GPU-Accelerated Tokenization for DNA Sequences with Memory-Latency Optimization

Thinh Nguyen[1], Yicheng Tu[1]

[1] Bellini College of Artificial Intelligence, Cybersecurity and Computing, University of South Florida, Tampa, FL 33620, USA

Motivation

- ✓ Machine learning (ML) is emerging in the Next-Generation Sequencing (NGS) field, demanding scalable tokenization tool for DNA sequences.
- ✓ Major tokenizers are CPU-only, while tokenization is a massively parallel task, making it well suited for GPU execution. However, naïve GPU implementations often suffer from memory inefficiencies.
- ✓ Our work aims to design a memory-centric GPU tokenizer that: eliminates shared-memory bank conflicts, maximizes coalesced memory access and memory bandwidth via minimal bit length representations.

NGS embedding workflow

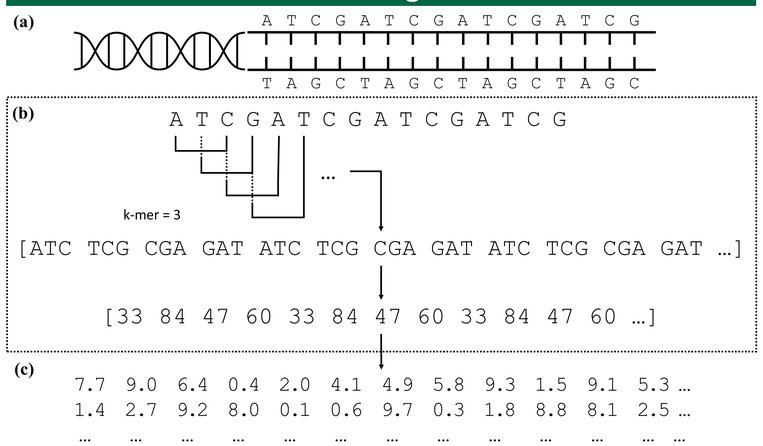

Figure 1. NGS embedding workflow. **(a)** Input sequence resulted from DNA sequencing. **(b)** Tokenization and indexing: The decomposition of the sequence into overlapping k-mers (where k=3) and their subsequent mapping to unique integer identifiers. **(c)** The projection of integer tokens into a high-dimensional vector space for downstream machine learning tasks.

GPU memory architecture

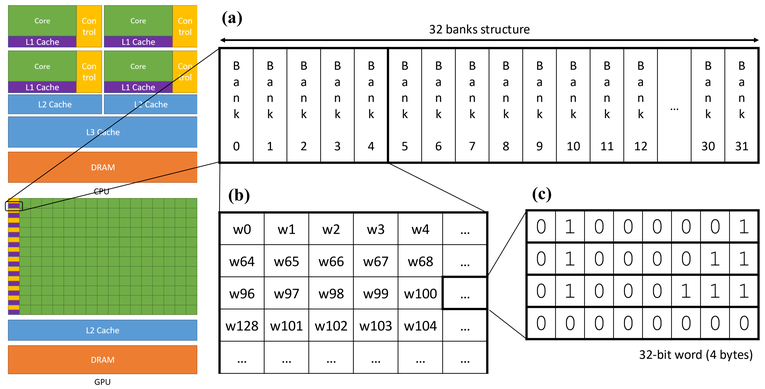

Figure 2. (left) High-level architectural comparison of CPU and GPU layouts[1]; **(right)** a detailed breakdown of **(a)** the 32-bank shared memory structure within a GPU thread block, **(b)** illustrating word distribution, and **(c)** bit-level representation.

[1] "1.1. Introduction — CUDA Programming Guide." https://docs.nvidia.com/cuda/cuda-programming-guide/01-introduction/introduction.html

Design

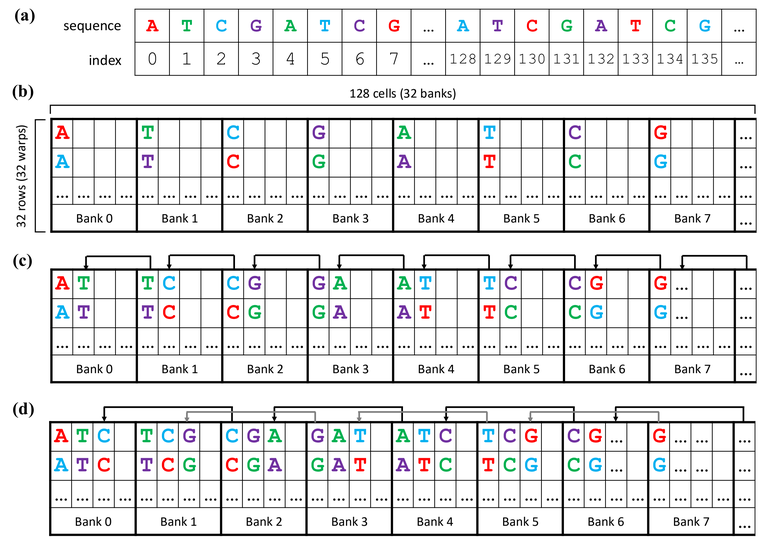

Figure 3. Banks conflict-free memory layout and operations for 3-mers tokenization. **(a)** DNA sequence residing in global memory. **(b)** Coalesced reading from global memory and zero-bank-conflict write pattern across 32 shared memory banks. **(c-d)** Shared memory's internal zero-bank-conflict population to complete a 3-mer token in each word block.

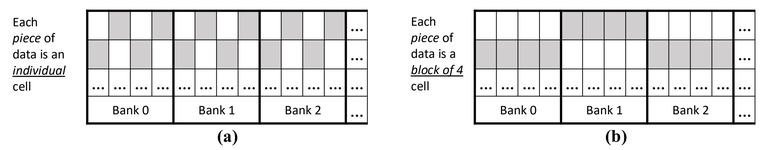

Figure 4. Reinterpret casting in shared memory. **(a)** Pre-cast layout with data accessed as individual 1-byte character cells. **(b)** Post-cast layout where data is accessed as 4-byte word blocks.

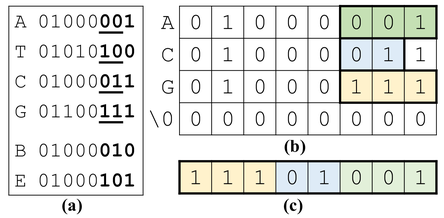

Figure 5. Bitwise compression mapping of 32-bit packed 3-mers. **(a)** ASCII binary representations of A, T, C, G, and sequence sentinels B-E, which were intentionally selected because their specific ASCII bit patterns provide uniquely distinguishable lower bits, preventing hash collisions during compression. **(b)** Bit-extraction mask applied to a 32-bit casted word. **(c)** The finalized 8-bit compressed token.

Token	Original value (32-bit)	Compressed value (8-bit)	Token's ID for embedder
BAA	4342081	34	0
BAC	4342083	98	1
…	…	…	…
ACG	4277063	233	24
…	…	…	…
TTT	5526612	148	95

(a) Token's ID & compressed value equivalence

Build a LUT using these known relationships

sorted

Index	Token's ID for embedder	Index	Token's ID for embedder
0	N/A (no valid token)	…	…
1	N/A (no valid token)	148	95 (TTT)
…	…	…	…
34	24 (BAA)	233	24 (ACG)
…	…	…	…
98	1 (BAC)	255	69 (GGG)

(b) LUT built at compile time

Figure 6. Compile-time Lookup Table (LUT) construction. **(a)** Mapping of 32-bit 3-mers to 8-bit compressed hashes, and target token IDs. **(b)** The LUT uses the 8-bit hash as a direct index to retrieve valid token IDs, ensuring collision-free *O(1)* lookups.

Benchmarks

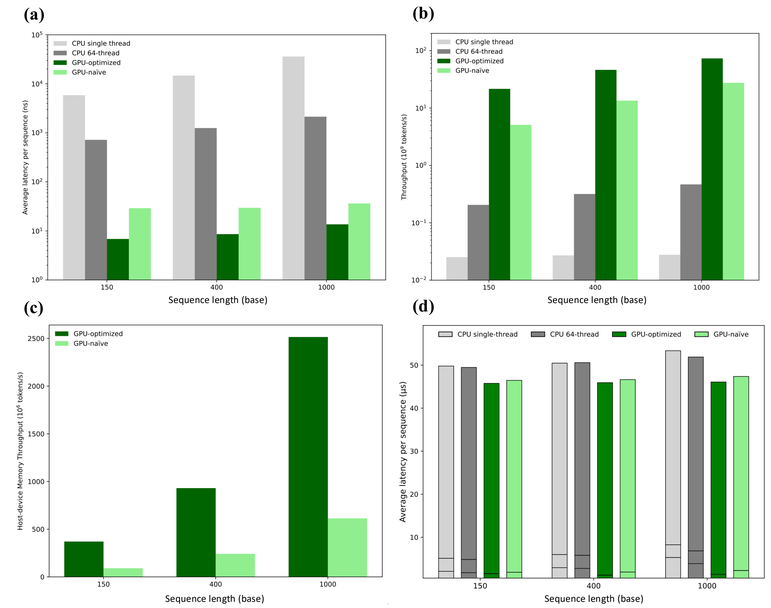

Figure 7. Performance comparison of four tokenization pipelines: CPU single-thread (light gray), CPU 64-thread (dark gray), GPU-optimized (dark green), and GPU-naïve (light green). Experiments were conducted on three datasets containing 100,000 sequences each with lengths of 150, 400, and 1000 bases. **(a)** Average latency per sequence (ns, log scale). **(b)** Tokenization throughput (billion tokens/s, log scale). **(c)** Host–device memory throughput (million tokens/s). **(d)** Latency breakdown of the GPU-only inference pipeline using each tokenizer, with stacked contributions from tokenization, host–device transfer, and inference.

Scaling and Limitations

The current implementation supports sequences up to 1024 bp and k-mer tokenization levels k=1 to 3. Using a 16-bit post-compression representation would extend the supported range to k=7. Further scaling requires redesign of shared-memory layout and global-memory batching method.

GPU implementations also incur host–device transfer overhead, making them most effective in end-to-end GPU pipelines rather than as accelerators for CPU-centric workflows.

Conclusions

- ✓ This work presents an efficient GPU tokenizer that outperforms both CPU-only pipelines and naïve GPU implementations, proving GPUs are well suited for tokenization task with their massively parallel architecture.
- ✓ Bit-length compression improves memory throughput by reducing token representation size while preserving information. These results suggest that lossless token compression is feasible and could be applied to other sequence-processing pipelines.
- ✓ Furthermore, this work demonstrates the efficiency of on-device tokenization within GPU inference pipelines, opening new opportunities for fully GPU-resident NGS and ML workflows.

Contact

Prof. Yicheng Tu
Bellini College of Artificial Intelligence, Cybersecurity and Computing, USF
Email: tuy@usf.edu
Website: https://cse.usf.edu/~tuy/

Source code

Let's connect!

UNIVERSITY of SOUTH FLORIDA